Airbnb európai szálláshelyek elemzése

Generatív AI és üzleti alkalmazások modul

dr. Pál László, Sapientia EMTE, 2026. március 6–7.

Blokk	Tartalom	Időtartam
1. feladat	Adatbetöltés	∼10 perc
1.5 feladat	EDA (Exploratory Data Analysis) – adatfeltáró elemzés	∼10 perc
2–3. feladat	Alapstatisztikák, outlier szűrés, korreláció-heatmap	∼20 perc
4–5. feladat	KDE, összefüggés-vizsgálatok	∼25 perc
6. feladat	K-means klaszterezés – Budapest	∼30 perc
7. feladat	Ár-előrejelzés – Random Forest	∼20 perc
8. feladat	Hasonló szállások ajánlója (opcionális)	∼15 perc
Teljes blokk		**∼115 perc**

Eszközök: Google Colab + Gemini (`gemini.google.com`)
Dataset: `airbnb_europe.csv`

Oszlopnevek: Amikor Gemini-nek promptot írsz, mindig pontosan ezeket az oszlopneveket add meg (nagy/kis betű és szóközök számítanak):

`Price, City, Day, Room Type, Superhost, Person Capacity, Cleanliness Rating, Guest Satisfaction, Bedrooms, City Center (km), Metro Distance (km), Normalised Attraction Index`

1. feladat – Adatbetöltés

Töltsd be az adatfájlt és vizsgáld meg az alapinformációkat. Az `airbnb_europe.csv` fájlt töltsd fel a Colab session-be (bal oldali fájlkezelő – feltöltés).

Prompt – adatbetöltés

```
Írj Python kódot Google Colab notebookban, amely:
- Betölti az airbnb_europe.csv fájlt pandas DataFrame-be (df)
- Kiírja a sorok és oszlopok számát
- Megjeleníti az első 5 sort (df.head())
- Kiírja az összes egyedi város nevét
- Ellenőrzi és kiírja a hiányzó értékeket oszloponként
```

1.5 feladat – EDA (Exploratory Data Analysis) – adatfeltáró elemzés

Mielőtt bármit szűrünk vagy transzformálunk, nézzük meg, mit tartalmaz az adatbázis. Ez a két diagram megmutatja, miért lesz szükség outlier szűrésre és log-transzformációra a következő lépésekben.

Prompt – EDA vizualizáció

```
Írj Python kódot, amely:
- Barplot: átlagos 'Price' városonként (sns.barplot, palette='Set3')
```

```
  Rendezd csökkenő sorrendbe az átlagár szerint
  Méret: 10x5, cím: 'Átlagos éjszakai ár városonként (EUR)'
  x tengelyen forgatás: 30 fok
- Hisztogram: a 'Price' oszlop nyers eloszlása (bins=60, color='steelblue')
  Méret: 10x4, cím: 'Ár eloszlása (nyers adatok)'
  x tengely felirat: 'Ár (EUR/éj)'

Meglévő változó: df (pandas DataFrame)
```

Tipp: A barplotból látszik, melyik városok drágábbak (pl. Amsterdam, Párizs vs. Budapest). A hisztogramból látszik az erős jobbra ferdülés – néhány szálláshely extrém magas ára torzítja az egész képet. Ez magyarázza, miért kell majd outlier szűrés és log-transzformáció.

2. feladat – Alapstatisztikák és outlier szűrés

Vizsgáljuk meg az ár eloszlását és a leíró statisztikákat. Ha a `describe()` eredménye alapján extrém kiugró értékek láthatók, szűrjük ki őket egy megfelelő percentilis határral.

Prompt – outlier szűrés

```
Írj Python kódot, amely:
- Kiírja a df.describe() eredményét (alapstatisztikák)
- Rajzolja meg a 'Price' oszlop hisztogramját
- Ha a hisztogram alapján szükséges, szűrje ki a kiugró
  értékeket a 'Price' oszlopból egy megfelelő percentilis határral
- Írja ki, hány sor maradt és mi volt a szűrési határ (EUR/éj)

Meglévő változó: df (pandas DataFrame)
Oszlopnév: 'Price'
```

Figyelem: Az outlier szűrés mértéke befolyásolja az összes eredményt – a klaszterközéppontokat, a modell pontosságát és az ajánló kimenetét is.

3. feladat – Korreláció-heatmap

A heatmap megmutatja, melyik változók függnek össze egymással. Az értékek −1 és +1 között mozognak: a kék cellák pozitív, a piros cellák negatív összefüggést jeleznek.

Prompt – korreláció-heatmap

```
Írj Python kódot, amely elkészíti a korreláció-heatmapet:
- Csak numerikus oszlopokat szerepeltess (df.select_dtypes)
- sns.heatmap, annot=True, cmap='RdBu_r', vmin=-1, vmax=1
- linewidths=0.5, linecolor='black'
- Méret: 14x9
- Cím: 'Korrelációs mátrix'

Meglévő változó: df (szűrt DataFrame)
```

Tipp: Figyeld meg a legerősebb összefüggéseket: `Cleanliness Rating`–`Guest Satisfaction` (0.69), `Person Capacity`–`Bedrooms` (0.56). Meglepetés: a `Price` és a `Guest Satisfaction` között nincs erős korreláció (0.02) – a drágább szállás nem jelent jobb élményt. Az `Attraction Index` és `Restaurant Index` változók egymással erősen korrelálnak (0.74–0.84): ugyanazt a jelenséget mérik különböző normalizálással. Ez a Random Forest feature importance-nál is látszani fog – a "szavazatok" megoszlanak közöttük.

4. feladat – Ár eloszlás városonként (log-transzformáció)

A KDE (Kernel Density Estimation) egy simított hisztogram: folytonos görbével mutatja meg, hol sűrűsödnek az árak. Több város egyszerre jól összehasonlítható rajta. A nyers ár eloszlása miatt azonban a városok görbéi alig különböztethetők meg – néhány extrém drága szálláshely szétnyújtja a skálát. A log10-transzformáció összenyomja a nagy értékeket, így az eloszlások összehasonlíthatóvá válnak. Például: 100 EUR → 2.0, 1000 EUR → 3.0.

Prompt – log-transzformált KDE

```
Írj Python kódot, amely:
- Létrehoz egy df_log másolatot a df-ből
- A df_log['Price'] oszlopot felváltja np.log10(df['Price'])-vel
- sns.displot KDE diagramot készít:
  x='Price', hue='City', kind='kde', fill=True
  height=6, aspect=1.8
- Cím: 'Ár eloszlás városonként (log10 skála)'
- x tengely felirat: 'log10(Ár)'

Meglévő változók: df, df_log
```

Tipp: A görbék csúcsa mutatja a leggyakoribb árkategóriát városonként: log10(2.0) = 100 EUR, log10(2.3) ≈ 200 EUR, log10(2.5) ≈ 316 EUR. Rome csúcsa a legszűkebb és legmagasabb – az árak kevésbé szóródnak. Amsterdam és Paris görbéje jobbra tolódik és lapos – drágább és változatosabb árszintű piac. Athens és Budapest balra helyezkedik – jellemzően olcsóbb szállások. A 3.0 feletti értékek (1000 EUR+) minden városban ritkaságok, de Amsterdam esetén a legsűrűbbek.

5. feladat – Összefüggés-vizsgálatok (binning)

Három kérdést vizsgálunk boxplot diagramokkal. A binning technika a folytonos változókat (pl. távolság km-ben) kategóriákra osztja, így boxplottal vizualizálhatók.

1. diagram – Tisztaság és vendégelégedettség

Prompt – cleanliness vs. elégedettség

```
Írj Python kódot seaborn boxplothoz:
- x='Cleanliness Rating', y='Guest Satisfaction'
- palette='Set3', width=0.7, whis=5
- Méret: 12x5
- Cím: 'Tisztaság hatása a vendégelégedettségre'
```

```
Meglévő változó: df
Oszlopnevek: 'Cleanliness Rating', 'Guest Satisfaction'
```

2. diagram – Városközponttól való távolság és ár

Prompt – távolság vs. ár

```
Írj Python kódot, amely:
- pd.cut segítségével a 'City Center (km)' oszlopot
  kategóriákra osztja: 0-3, 3-6, 6-9, ..., 24-27 km
- Menti a df['dist_bin'] oszlopba
- sns.boxplot: x='dist_bin', y=np.log10(df['Price'])
  palette='Set3', width=0.7
- Méret: 12x5, x tengelyen 30 fokos forgatás
- Cím: 'Városközpont távolsága vs. ár (log10)'

Meglévő változók: df, df_log
Oszlopnevek: 'City Center (km)', 'Price'
```

3. diagram – Vendégelégedettség és ár

Prompt – elégedettség vs. ár

```
Írj Python kódot, amely:
- pd.cut segítségével a 'Guest Satisfaction' oszlopot
  10-es lépésekkel osztja kategóriákra: 0-10, 10-20, ..., 90-100
- Menti a df['sat_bin'] oszlopba
- sns.boxplot: x='sat_bin', y=np.log10(df['Price'])
  palette='Set3', width=0.7
- Méret: 12x5
- Cím: 'Vendégelégedettség vs. ár (log10)'

Meglévő változók: df, df_log
```

Tipp: 1. diagram: egyértelmű összefüggés – a tisztább szálláshely magasabb vendégelégedettséggel jár. 2. diagram: az ár a városközponttól távolodva enyhe csökkenést mutat, de a különbség kisebb a vártnál – log10(2.3) $\approx$ 200 EUR vs. log10(2.2) $\approx$ 158 EUR. A 21–24 km-es kategória kilóg: valószínűleg repülőtér-közeli szállások emelik az átlagot. 3. diagram: a `Guest Satisfaction` és az ár között nincs érdemi összefüggés – a drágább szállás nem jelent jobb értékelést, és fordítva sem.

6. feladat – K-means klaszterezés (Budapest)

A K-means tanítás nélküli (unsupervised) algoritmus: előre megcímkézett adat nélkül fedez fel csoportokat a hasonlóságok alapján. Egyszerre veszi figyelembe az összes változót – ár, távolság, elégedettség, hálószobák száma stb. – így üzleti szegmensek azonosíthatók (pl. budget, prémium, nagy kapacitású). A folytonos változókat standardizálni kell, mert eltérő skálán vannak; a kategorikus oszlopokat one-hot encodinggal bináris (0/1) értékekké alakítjuk.

6.1 – Adatelőkészítés és elbow method

Prompt – Budapest szűrés és elbow

```
Írj Python kódot, amely:
- Szűri a budapesti adatokat: df_bp = df[df['City'] == 'Budapest'].copy()
- One-hot encoding a kategorikus oszlopokra (pd.get_dummies)
- Bool oszlopokat int8-ra konvertál
- StandardScaler-rel standardizálja a numerikus oszlopokat (X_scaled)
- Elbow method: KMeans K=2..15, inertia értékeket gyűjt és kirajzolja
  x='k', y='Inertia', marker='o'
- Cím: 'Elbow method -- optimális K meghatározása'

Meglévő változó: df
```

Figyelem: Ennél a görbénél nincs éles törés – ez tipikus. K=4 és K=6 között érdemes kísérletezni: futtasd le mindkettőt és azt válaszd, amelyik klaszterprofil üzletileg értelmesebb (pl. budget, prémium, nagy kapacitású szegmens).

6.2 – Klaszterezés és PCA vizualizáció

A PCA (Principal Component Analysis) az összes változó információját sűríti 2 dimenzióba, hogy vizualizálható legyen. A tengelyértékek önmagukban nem értelmezhetők – csak a pontok egymáshoz viszonyított távolsága számít: a közel lévő pontok hasonló jellemzőjű szállásokat jelentenek. Az a lényeg, hogy a ***színek szétválnak-e****: ez jelzi, hogy az algoritmus valódi csoportokat talált.*

Prompt – K-means és PCA

```
Írj Python kódot, amely:
- KMeans(n_clusters=5, n_init='auto', random_state=42) futtat X_scaled-en
- A klaszter labeleket hozzáadja df_bp['cluster'] oszlopként
- Kiírja a klaszter profilokat: csoportonkénti átlagok
  'Price', 'Guest Satisfaction', 'City Center (km)', 'Bedrooms' oszlopokra
- PCA(n_components=2) projekció, scatter plot klaszterszín szerint
  (cmap='tab10', s=20, alpha=0.7)
- Cím: 'Budapesti szállások klaszterei (PCA projekció)'

Meglévő változók: df_bp, X_scaled
```

Tipp: A profilok üzleti szegmenseket mutatnak: a legdrágább, legjobb elhelyezkedésű klaszter a prémium szegmens, a magas távolság + alacsony ár kombináció a budget külvárosi szegmens, a nagy tömegű középső klaszterek különböző árszintű átlagos szállásokat jelentenek. A PCA diagramon a színek szétválása megerősíti, hogy az algoritmus valódi csoportokat talált.

7. feladat – Ár-előrejelzés (Random Forest, Budapest)

A Random Forest döntési fák sokaságából álló ensemble modell. Most arra tanítjuk, hogy a szálláshely jellemzői alapján megjósolja az árat. A feature importance megmutatja, melyik változó befolyásolja leginkább az árat.

7.1 – Modell tanítása és értékelése

Prompt – Random Forest modell

```
Írj Python kódot, amely:
- df_bp-ből a célváltozó: y = df_bp['Price']
- Feature-ök: X = df_bp összes többi numerikus oszlopa
- Train/test split: 80/20, random_state=42
- RandomForestRegressor(n_estimators=500, random_state=42, n_jobs=-1)
- Kiírja Train és Test MAE, RMSE, R2 értékeket
- Top 15 feature importance vízszintes barplot
  Cím: 'Top 15 -- mi határozza meg az árat Budapesten?'

Meglévő változók: df_bp (one-hot kódolt DataFrame)
```

Tipp: Test R^2=0.60: a modell az árvarianciának 60%-át magyarázza – a maradék 40% olyan tényezőktől függ, amelyek nincsenek az adatban (fotók minősége, leírás stílusa, gazda válaszkészsége). A Train/Test R^2 különbség (0.91 vs. 0.60) túlilleszkedést jelez.
Feature importance: az elhelyezkedés (`Metro Distance`, `City Center`) dominálja az árat – ez összhangban van az 5. feladat boxplot eredményeivel. Az `Attraction` és `Restaurant` indexek együtt ∼40%-ot tesznek ki: pontosan a 3. feladatnál jelzett szavazatmegoszlás. Az EDA-ban látott összefüggések tehát visszaigazolódnak a gépi tanulási modellben.

8. feladat – Hasonló szállások ajánlója (opcionális)

Ez a feladat opcionális – ha az idő engedi, vagy házi projektként. Elvégzése nem feltétele a többi feladatnak.

A **cosine similarity** két vektor közötti szög koszinuszát méri. Ha két szálláshely numerikus jellemzői hasonlók, a köztük lévő szög kicsi – a hasonlóság értéke 1-hez közelít. Az ajánló az összes városra dolgozik, így előfordulhat, hogy egy budapesti szálláshely mellé bécsi kerül – ez nem hiba, hanem az azonos ár-érték profil következménye.

Továbbfejlesztés – beágyazásos ajánló (embedding-alapú): A mostani megközelítés numerikus jellemzőkön dolgozik (ár, távolság stb.). A modern ajánlórendszerek ennél tovább mennek: a szálláshely szöveges leírásából nyelvi modell segítségével **beágyazási vektorokat** (embeddings) generálnak, és a szemantikai hasonlóságot mérik – nem csak a számokat. Ehhez pl. a `sentence-transformers` könyvtár használható Colabon, API kulcs nélkül (`paraphrase-multilingual-MiniLM` modell). Ez a házi projekt egyik lehetséges iránya.

Prompt – cosine similarity ajánló

```
Írj Python kódot, amely felépíti a szállásajánlót:
- df-ből válaszd ki ezeket a feature-öket:
  'Price', 'Guest Satisfaction', 'City Center (km)',
  'Bedrooms', 'Normalised Attraction Index', 'Metro Distance (km)'
- StandardScaler standardizálás
- Írj ajanlj(idx, top_n=5) függvényt, amely:
  - cosine_similarity segítségével megkeresi a top_n
    leghasonlóbb szállást (saját magát kizárva)
  - Kiírja a referencia szállást és a top 5 ajánlást:
    'City', 'Room Type', 'Price', 'City Center (km)',
    'Guest Satisfaction', hasonlóság értéke
```

```
- Teszteld egy budapesti és egy párizsi szállásra

Szükséges import: from sklearn.metrics.pairwise import cosine_similarity
Meglévő változó: df (teljes, 10 városos DataFrame)
```

Tipp: Az ajánló városok között is keres – egy budapesti prémium szállás mellé bécsi kerülhet, mert az ár-érték profiljuk hasonló. Ez a módszer korlátja is: csak a numerikus jellemzőket látja, a város karakterét, a nyelvet vagy a kulturális kontextust nem. Pontosan ezért mutat tovább az embedding-alapú megközelítés.

Cheat Sheet – Airbnb elemzés

Adatbetöltés és tisztítás

`pd.read_csv('airbnb_europe.csv')`	fájl betöltése
`df.isnull().sum()`	hiányzó értékek
`df['Price'].quantile(0.99)`	99. percentilis
`df[df['Price'] <= p99]`	outlier szűrés
`np.log10(df['Price'])`	log-transzformáció
`pd.cut(df[x], bins=..., labels=...)`	binning

K-means klaszterezés – lépések

1. Kategorikus oszlopok: `pd.get_dummies`
2. Standardizálás: `StandardScaler().fit_transform(X)`
3. Elbow: K=2..15, inertia görbe, keresd a töréspontot
4. `KMeans(n_clusters=K, n_init='auto', random_state=42)`
5. Profilok: `df.groupby('cluster')[features].mean()`
6. PCA vizualizáció: `PCA(n_components=2)`

Random Forest – lépések

1. `train_test_split(X, y, test_size=0.2)`
2. `RandomForestRegressor(n_estimators=500, n_jobs=-1)`
3. Metrikák: MAE, RMSE, R2 – train és test halmazon
4. `rf.feature_importances_` – melyik változó számít?

Cosine similarity ajánló (opcionális)

1. Feature-ök standardizálása
2. `cosine_similarity(matrix[i].reshape(1,-1), matrix)`
3. Saját magát kizárjuk: `sim[idx] = 0`
4. Top N: `sim.argsort()[::-1][:n]`

\+ Továbbfejlesztés: `sentence-transformers` szöveges beágyazással

Vibe coding – hasznos promptminták

Adatbetöltés	*"Írj kódot, amely betölti a CSV-t és kiírja az alapinformációkat"*
Vizualizáció	*"Adj hozzá tengelyfeliratot, címet és rácsozást"*
Hiba esetén	*"Ez a hibaüzenet jelent meg: [...] Javítsd ki"*
Finomítás	*"A diagram túl zsúfolt, csökkentsd a betűméretet"*
Magyarázat	*"Magyarázd el mit csinál ez a kódrészlet"*